

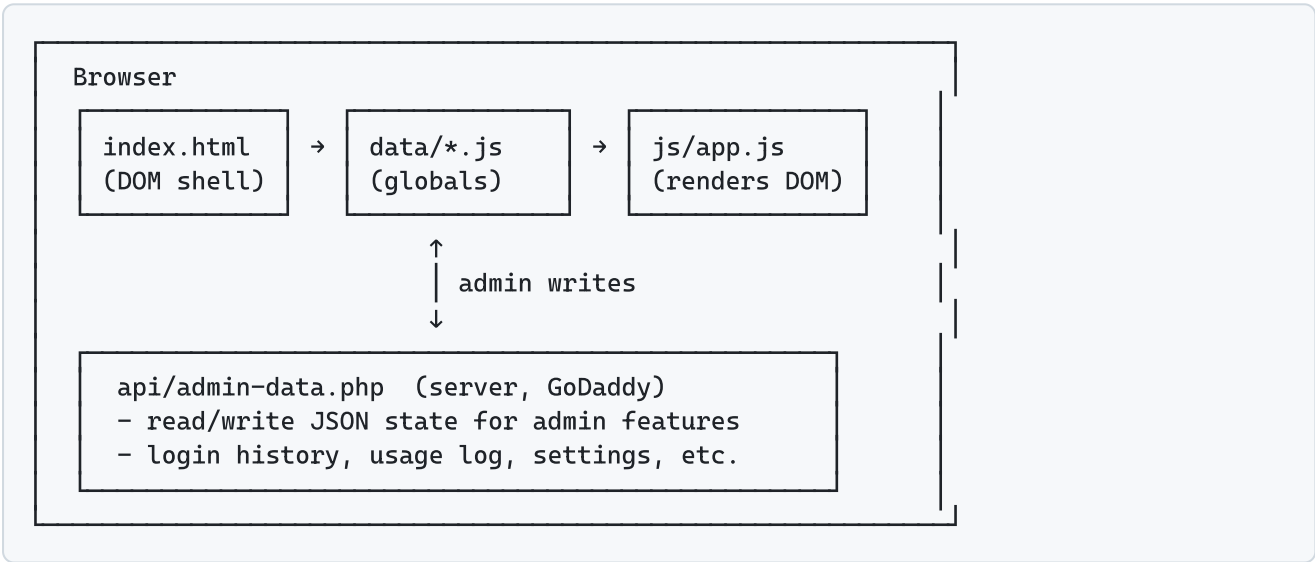
Technical Specification

Overview

CDTA 2026 Dashboard is a **single-page client-rendered web app** with no build step. It is plain HTML + vanilla JavaScript + CSS, served as static files from GoDaddy. A small PHP backend (`api/admin-data.php`) provides persistence for admin-modified data.

There is no framework, no bundler, and no transpiler. All logic lives in `js/app.js` (~5,200 lines).

Architecture



Load order

- `index.html` loads all 11 `data/division-{a..k}.js` files plus `data/auth.js` as `<script>` tags. Each file declares globals like `DIV_C_PLAYERS`, `DIV_C_SCHEDULE`, `DIV_C_PARTNERS`.
- `js/app.js` runs after all data is loaded. It defines render functions and immediately calls them at the bottom of the file.
- Each division has its own init block (`renderDivXStandings()`, `renderDivXTeams()`, etc.).

Key modules in `js/app.js`

Section	Lines (approx)	What it does
Helpers (<code>_esc</code> , <code>_standingRowClass</code> , etc.)	top	Shared utilities
Authentication & menu	1–400	Login, role enforcement, menu visibility

Section	Lines (approx)	What it does
Division I (default)	400–800	The original division — <code>renderStandings()</code> , <code>renderPlayers()</code> , <code>renderResults()</code> , etc.
Division C	600–3000	Manual division with projections, key pairs, partners
<code>recomputeDivStats()</code>	~3070	Critical helper — derives player stats from schedule courts
Divisions F, H	3080–3700	Manual divisions
Divisions A, B, D, E, G, J, K	3700–4500	Scraped divisions (rendering only — data is pre-loaded)
Admin panel	1500–2700	User mgmt, logs, data tools, integrity check, auto-fix
Overview	900–1300	Cross-division consolidation
SVG Players club view	2900–3060	Special club view

The recompute pipeline (most important algorithm)

After a season refresh, the static `gamesPlayed` and `wins` fields in `DIV_X_PLAYERS` may be stale or `null` . Rather than store these values, **they are derived at runtime** from the authoritative source — the per-court records in `DIV_X_SCHEDULE` .

Function: `recomputeDivStats(SCHEDULE, PLAYERS, PARTNERS, KEY_PAIRS?)`

Located in `js/app.js` , called once per manual division (C, F, H, I) before any render.

Steps: 1. Build a name normalization map: `normalize(name)` → strip dots/commas/apostrophes, uppercase, collapse whitespace. 2. Build canonical name lookup: `normalized` → `display_name` from `PLAYERS` . 3. Walk every `SCHEDULE[].matches[].courts[]` : - Split `home` and `away` strings on `/` to get the two-player pair - For each player, increment `played` ; if their side won, increment `wins` - Track partner counts: each player's other-side-of-the-slash partner gets `+1` - Track pair stats keyed by sorted `(p1|p2)` tuple → `{apps, wins}` 4. Assign `gamesPlayed` / `wins` back to `PLAYERS` items by normalized-name lookup 5. Rebuild `PARTNERS` object: for each player, list `[partner_display, count]` sorted desc 6. If `KEY_PAIRS` is provided: for each entry, parse `pair` string, look up in pair-stats, fill in `apps` and `wins`

Name normalization rule

```
norm(s) = s.toUpperCase()
          .replace(/[.,'`]/g, ' ') // strip punctuation
          .replace(/\s+/g, ' ')   // collapse spaces
          .trim();
```

This handles cases like `J.SURESH KUMAR` (schedule) ↔ `J. Suresh Kumar` (roster) — both normalize to `J SURESH KUMAR` .

Standings calculation

Each division has its own `renderDiv{X}Standings()` function. The shared logic:

```
const playedPts = s.played;           // courts played = max possible pts
const matches   = Math.floor(s.played / 3);
const ptsPct    = Math.round((s.pts / playedPts) * 100);
```

Exception — Division K: The K data file stores `played` as **matches** (not courts), because the season was scraped late and the source page exposes match counts only. Its render compensates:

```
const matches   = s.played;
const playedPts = s.played * 3;
```

Data flow for a weekly refresh

```
league.cdta.co.in
  ↓ (manual scrape via /load-division skill,
    or admin Data Tools → Auto Fix Stats)
data/division-{a..k}.js ← regenerated, in courts
  ↓
git commit + push
  ↓
deploy.bat / deploy.ps1 ← FTP to GoDaddy
  ↓
http://akilanramesh.com/cdta/
```

Admin persistence (server-side)

`api/admin-data.php` is a single PHP endpoint storing JSON files on the GoDaddy server. It is called from the admin panel for:

- User management (`auth.json` mirror)
- Login history append
- Usage log append
- Activity log append
- Settings updates

Auth in PHP is shared-secret based — only the admin role's session token can write.

Browser support

- Modern Chrome / Edge / Safari / Firefox
 - Uses CSS Grid, Flexbox, ES2017+ (template literals, arrow functions, optional chaining)
 - No polyfills, no IE support
-

Performance notes

- **Cold load:** ~1.5–3 MB JS payload (mostly the 11 division data files), no minification. Acceptable for an internal tool.
 - **Photos:** lazy-loaded by browser; ~64 MB total but each `<img>` only fetches when visible.
 - **No virtual DOM:** render functions rebuild `innerHTML` of their target table on every filter change. Tables max ~125 rows so this is fine.
 - **Sorting:** in-place array sort, then full re-render.
-

Known technical debt

- A single 5,200-line `app.js` is hard to navigate. Future split could be by division.
- Several lint hints for unused variables (`totalPlayed` , `stand` , `teams` , etc.) — pre-existing, harmless.
- Player name normalization is rule-based; rare unicode names may slip through.
- No automated tests — only the manual Test Plan in the admin panel.